Paragliding Intelligence Platform

Volume 03 - Database Design Document v2

Implementation-grade PostgreSQL, PostGIS and TimescaleDB schema design

Field	Value
Document type	Database Design Document (DDD)
Architecture baseline	PRD v2 Technical + SAD v2 Technical
Primary database	PostgreSQL with PostGIS and TimescaleDB
Primary goal	Support a next-edge paragliding meteo decision platform: flyability, risk, thermals, digital twins, AI briefing, tracking, club/school/event modules and paid entitlements
Version	v2 technical
Generated	2026-06-04

1. Executive Summary

This document replaces the earlier high-level placeholders with a concrete database design for the paragliding meteo platform. The database must support weather forecast ingestion, geospatial launch data, terrain-derived models, flyability and risk scoring, AI grounded briefings, live tracking, subscriptions, clubs, schools, competitions and operational observability.

The architecture uses PostgreSQL as the system of record, PostGIS for spatial objects and queries, TimescaleDB hypertables for high-frequency time-series, Redis for hot cache, and object storage for large binary source files such as GRIB, NetCDF, DEM rasters, IGC files and map tile packages.

Design decision	Reason
PostgreSQL core store	Strong relational integrity for users, launches, subscriptions, clubs, schools, competitions, alert rules and audit data.
PostGIS geometry/geography	Launches, landings, airspaces, terrain polygons, hazard zones, thermal trigger zones and nearest-site queries need spatial types and GiST indexes.
TimescaleDB hypertables	Forecasts, station observations, pilot observations, track points and score history are append-heavy time-series datasets.
Object storage references	Raw GRIB/NetCDF/DEM/IGC files are too large and too immutable for relational storage; the DB stores metadata and object URLs/checksums.
Event-outbox tables	Critical notifications and downstream ML jobs must be reliable even if Kafka/queue workers are temporarily unavailable.
Entitlement tables	Paid tiers must control feature gates, forecast ranges, API quotas, AI credits and enterprise modules.

2. Database Architecture Principles

- The database is feature-oriented, not vendor-oriented. Weather providers are normalized into common model/run/grid/forecast tables.
- Every time-series table must have an explicit time column, hypertable strategy, retention policy and query index plan.
- Every spatial table must explicitly define SRID, geometry type, coordinate semantics and spatial index strategy.
- Every user-facing recommendation must be reproducible from a stored context snapshot, source forecast references and score factor records.
- No AI decision is stored as a black box. The database stores the structured context, score inputs, hard blockers, model version and generated response hash.
- The app separates raw weather truth from corrected site-level predictions. Raw forecasts remain immutable after ingestion; corrections are versioned.
- Paid features are controlled by entitlements, not client-side flags.
- PII is isolated and auditable. Location tracking, rescue, billing and communications data have stricter retention and access controls.

3. External Data Source Baseline

The schema is designed to ingest and normalize multiple providers. The DDD does not depend on a single forecast API. It supports API-based MVP sources and raw GRIB/DEM/airspace ingestion for advanced tiers.

Source type	Provider examples	Database impact
Multi-model weather API	Open-Meteo	Used for MVP site forecasts and as normalized forecast inputs. Store provider/model/run metadata and hourly values.
Nordic forecast API	MET Norway Locationforecast	Used for Norway/Nordics site forecasts; store API payload metadata and normalized variables.
Raw global forecast model	NOAA GFS via NOMADS grib filter	Requires model run metadata, GRIB object references, grid cells and extracted variables.
Raw premium/global forecast model	ECMWF open/premium datasets	Same normalized model/run/grid structure; attribution/license stored at provider level.
Airspace data	OpenAIP	Requires polygon/multipolygon airspace tables with class, floor/ceiling, activity windows and source revision.
Terrain data	Copernicus DEM / local DEM	Requires DEM tile metadata, raster-derived

		terrain cells, slope/aspect/roughness fields and thermal trigger zones.

4. Global Database Conventions

4.1 Schemas

Schema	Purpose	Owner service
auth	Accounts, users, sessions, roles, API keys	identity-service
billing	Plans, subscriptions, entitlements, usage metering	billing-service
catalog	Launch sites, landings, hazards, site rules, media, moderation	site-catalog-service
geo	Terrain, DEM metadata, airspace, map tiles, thermal trigger geometry	geo-service
weather	Providers, model runs, raw forecast cubes, site forecasts, observations	weather-ingestion-service / forecast-service
risk	Flyability scores, safety blockers, risk factors, decision snapshots	risk-engine-service
flight	IGC files, tracks, track points, live sessions	tracking-service
ai	Briefing requests, context snapshots, AI output metadata	ai-briefing-service
alerts	Alert rules, notification events, delivery attempts	alert-service
org	Clubs, schools, events, memberships, permissions	organization-service
ops	Ingestion jobs, health checks, data-quality checks, service state	ops-service
audit	Immutable security and data access events	audit-service

Bootstrap extensions and schemas

```
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS timescaledb;
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS btree_gist;
CREATE EXTENSION IF NOT EXISTS pg_trgm;

CREATE SCHEMA IF NOT EXISTS auth;
CREATE SCHEMA IF NOT EXISTS billing;
CREATE SCHEMA IF NOT EXISTS catalog;
CREATE SCHEMA IF NOT EXISTS geo;
CREATE SCHEMA IF NOT EXISTS weather;
CREATE SCHEMA IF NOT EXISTS risk;
CREATE SCHEMA IF NOT EXISTS flight;
CREATE SCHEMA IF NOT EXISTS ai;
CREATE SCHEMA IF NOT EXISTS alerts;
CREATE SCHEMA IF NOT EXISTS org;
CREATE SCHEMA IF NOT EXISTS ops;
CREATE SCHEMA IF NOT EXISTS audit;
```

4.2 Units, SRIDs and timestamps

Domain	Standard
Coordinates	EPSG:4326 WGS84. Use geography(Point,4326) for global distance queries; geometry for polygons and tile operations.
Altitude	Meters above mean sea level unless explicitly marked AGL.
Wind speed	m/s in database. Convert to km/h, knots or mph in the API/UI.
Wind direction	Degrees true, meteorological convention: direction wind comes from.
Temperature	Celsius.
Pressure	hPa.
Cloudbase	Meters MSL and optional AGL relative to site.
Time	TIMESTAMPTZ in UTC for all persisted event and forecast times.
JSON payloads	JSONB for provider payload snapshots, source metadata, AI context and extensible risk factor payloads.

5. Feature-to-Data Ownership Matrix

Feature	Primary tables	Owner service	Read dependencies
GO/MAYBE/NO-GO flyability	risk.flyability_hourly, risk.risk_factor_values, weather.site_forecast_hourly	risk-engine-service	catalog.launch_sites, catalog.launch_orientations, weather.model_runs, auth.pilot_profiles
XC planner	risk.xc_potential_hourly, geo.thermal_trigger_zones, weather.site_forecast_hourly, flight.landing_fields	xc-planner-service	terrain cells, route segments, cloudbase, wind profiles
AI flight briefing	ai.briefing_requests, ai.briefing_context_snapshots, ai.briefing_outputs	ai-briefing-service	risk scores, forecasts, launch rules, pilot profile, entitlements
Digital twin correction	weather.forecast_error_samples, weather.site_model_corrections, weather.corrected_site_forecast_hourly	digital-twin-service	station observations, pilot reports, historical scores
Live reality layer	weather.pilot_observations, weather.station_observations, flight.live_sessions	reality-service	launch site, user reputation, device telemetry
Alerts	alerts.alert_rules, alerts.alert_events, alerts.notification_attempts	alert-service	forecast deltas, risk scores, user subscriptions
Airspace guardian	geo.airspace_zones, flight.track_points, alerts.alert_events	airspace-service	live GPS, altitude, airspace schedule
Paid tiers	billing.plans, billing.entitlements, billing.subscriptions, billing.usage_meter	billing-service	auth.users, org memberships
Club/school dashboards	org.organizations, org.memberships, org.school_students, org.event_tasks	organization-service	sites, weather, risk, tracking
Competition mode	org.events, org.event_tasks, flight.tracks, flight.track_points	competition-service	weather, airspace, live tracking

6. Logical Entity Relationship Overview

```
auth.users 1--1 auth.pilot_profiles
users 1--N billing.subscriptions
users 1--N alerts.alert_rules
users 1--N flight.tracks
users N--N org.organizations via org.memberships

catalog.launch_sites 1--N catalog.launch_orientations
catalog.launch_sites 1--N catalog.landing_zones
catalog.launch_sites 1--N catalog.site_hazards
catalog.launch_sites 1--N weather.site_forecast_hourly
catalog.launch_sites 1--N risk.flyability_hourly
catalog.launch_sites 1--1 weather.site_digital_twins

weather.providers 1--N weather.weather_models
weather.weather_models 1--N weather.model_runs
weather.model_runs 1--N weather.forecast_grid_hourly
weather.model_runs 1--N weather.site_forecast_hourly

geo.dem_tiles 1--N geo.terrain_cells
geo.terrain_cells N--N catalog.launch_sites through proximity/materialized views
geo.airspace_zones N--N flight.track_points through spatial intersection

risk.flyability_hourly 1--N risk.risk_factor_values
ai.briefing_requests 1--1 ai.briefing_context_snapshots
ai.briefing_requests 1--1 ai.briefing_outputs
```

7. Auth, User and Pilot Profile Schema

User data is separated from pilot-specific operational data. Pilot profile fields are used by risk and recommendation engines but must remain editable, versioned and explainable.

```
CREATE TYPE auth.user_status AS ENUM ('active','disabled','pending_delete');
CREATE TYPE auth.pilot_level AS ENUM
('student','beginner','intermediate','advanced','xc','competition','instructor','tandem');
CREATE TYPE auth.wing_class AS ENUM ('EN_A','EN_B','EN_C','EN_D','CCC','speedwing','unknown');

CREATE TABLE auth.users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email CITEXT UNIQUE NOT NULL,
    display_name TEXT NOT NULL,
    status auth.user_status NOT NULL DEFAULT 'active',
    home_location GEOGRAPHY(POINT,4326),
    locale TEXT DEFAULT 'en',
    timezone TEXT DEFAULT 'UTC',
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    deleted_at TIMESTAMPTZ
);

CREATE TABLE auth.pilot_profiles (
    user_id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
    pilot_level auth.pilot_level NOT NULL DEFAULT 'beginner',
    wing_class auth.wing_class NOT NULL DEFAULT 'unknown',
    max_comfort_wind_ms NUMERIC(4,2),
    max_comfort_gust_ms NUMERIC(4,2),
    min_cloudbase_agl_m INTEGER,
    risk_tolerance SMALLINT NOT NULL DEFAULT 3 CHECK (risk_tolerance BETWEEN 1 AND 5),
    prefers_xc BOOLEAN NOT NULL DEFAULT false,
    prefers_hike_and_fly BOOLEAN NOT NULL DEFAULT false,
    medical_or_emergency_notes TEXT,
    emergency_contact JSONB,
    profile_version INTEGER NOT NULL DEFAULT 1,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX users_home_location_gix ON auth.users USING GIST(home_location);
CREATE INDEX pilot_profiles_level_idx ON auth.pilot_profiles(pilot_level, wing_class);
```

Field	Why it exists
max_comfort_wind_ms / max_comfort_gust_ms	Allows personalized safety thresholds instead of generic forecasts.
risk_tolerance	Controls recommendation wording and threshold margins. It does not override hard safety blockers.
profile_version	Stored with AI/risk snapshots so a past recommendation can be reproduced.
emergency_contact	JSONB avoids over-normalizing rare fields; encrypted-at-rest column-level encryption can be added later.

8. Launch Site Catalogue Schema

The launch catalogue is the core domain object. Weather is useless for paragliding without launch orientation, safe ranges, hazards and local rules.

```
CREATE TYPE catalog.site_status AS ENUM ('draft','active','closed','seasonal','restricted','archived');
CREATE TYPE catalog.site_difficulty AS ENUM ('student','beginner','intermediate','advanced','expert');
CREATE TYPE catalog.surface_type AS ENUM ('grass','dirt','rock','snow','sand','ramp','unknown');

CREATE TABLE catalog.launch_sites (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name TEXT NOT NULL,
    country_code CHAR(2) NOT NULL,
    region TEXT,
    status catalog.site_status NOT NULL DEFAULT 'draft',
    difficulty catalog.site_difficulty NOT NULL DEFAULT 'intermediate',
    launch_point GEOGRAPHY(POINT,4326) NOT NULL,
    launch_area GEOMETRY(POLYGON,4326),
    launch_altitude_m INTEGER NOT NULL,
    landing_altitude_m INTEGER,
    vertical_drop_m INTEGER GENERATED ALWAYS AS (launch_altitude_m - COALESCE(landing_altitude_m, launch_altitude_m))
STORED,
    surface catalog.surface_type DEFAULT 'unknown',
    default_min_wind_ms NUMERIC(4,2),
    default_max_wind_ms NUMERIC(4,2),
    default_max_gust_ms NUMERIC(4,2),
    min_cloudbase_agl_m INTEGER,
    description TEXT,
```

```
    local_rules TEXT,
    source TEXT NOT NULL DEFAULT 'community',
    source_url TEXT,
    confidence_score NUMERIC(4,3) DEFAULT 0.500,
    created_by UUID REFERENCES auth.users(id),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE catalog.launch_orientations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    launch_site_id UUID NOT NULL REFERENCES catalog.launch_sites(id) ON DELETE CASCADE,
    direction_center_deg SMALLINT NOT NULL CHECK (direction_center_deg BETWEEN 0 AND 359),
    direction_tolerance_deg SMALLINT NOT NULL CHECK (direction_tolerance_deg BETWEEN 5 AND 120),
    min_wind_ms NUMERIC(4,2) NOT NULL,
    max_wind_ms NUMERIC(4,2) NOT NULL,
    max_gust_ms NUMERIC(4,2) NOT NULL,
    notes TEXT,
    UNIQUE(launch_site_id, direction_center_deg)
);

CREATE TABLE catalog.landing_zones (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    launch_site_id UUID REFERENCES catalog.launch_sites(id) ON DELETE SET NULL,
    name TEXT NOT NULL,
    landing_point GEOGRAPHY(POINT,4326) NOT NULL,
    landing_area GEOMETRY(POLYGON,4326),
    altitude_m INTEGER,
    difficulty catalog.site_difficulty DEFAULT 'beginner',
    hazards TEXT,
    is_primary BOOLEAN DEFAULT false,
    created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE catalog.site_hazards (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    launch_site_id UUID NOT NULL REFERENCES catalog.launch_sites(id) ON DELETE CASCADE,
    hazard_type TEXT NOT NULL, -- power_line, rotor_zone, cliff, trees, road, cable, turbulence, no_landing
    severity SMALLINT NOT NULL CHECK (severity BETWEEN 1 AND 5),
    geometry GEOMETRY(GEOMETRY,4326),
    description TEXT NOT NULL,
    active_from DATE,
    active_to DATE,
    created_at TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX launch_sites_point_gix ON catalog.launch_sites USING GIST(launch_point);
CREATE INDEX launch_sites_area_gix ON catalog.launch_sites USING GIST(launch_area);
CREATE INDEX launch_sites_country_status_idx ON catalog.launch_sites(country_code, status);
CREATE INDEX launch_sites_name_trgm_idx ON catalog.launch_sites USING GIN(name gin_trgm_ops);
CREATE INDEX landing_zones_point_gix ON catalog.landing_zones USING GIST(landing_point);
CREATE INDEX site_hazards_geometry_gix ON catalog.site_hazards USING GIST(geometry);
```

Critical query: nearest launch search

```
-- Find active launches within 120 km of a user coordinate.
SELECT id, name, ST_Distance(launch_point, ST_MakePoint(:lon, :lat)::geography) AS distance_m
FROM catalog.launch_sites
WHERE status = 'active'
  AND ST_DWithin(launch_point, ST_MakePoint(:lon, :lat)::geography, 120000)
ORDER BY distance_m
LIMIT 50;
```

9. Terrain, Airspace and GIS Schema

GIS data supports launch suitability, thermal prediction, XC route planning and airspace safety. Raster source files stay in object storage; derived cells/zones are stored as relational/spatial features.

```
CREATE TABLE geo.dem_tiles (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    provider TEXT NOT NULL, -- Copernicus DEM, SRTM, local lidar
    resolution_m NUMERIC(6,2) NOT NULL,
    bounds GEOMETRY(POLYGON,4326) NOT NULL,
    object_uri TEXT NOT NULL,
    checksum_sha256 TEXT NOT NULL,
```

```
    acquired_at TIMESTAMPTZ,
    processed_at TIMESTAMPTZ,
    metadata JSONB DEFAULT '{}'
);

CREATE TABLE geo.terrain_cells (
    id BIGSERIAL PRIMARY KEY,
    dem_tile_id UUID REFERENCES geo.dem_tiles(id) ON DELETE CASCADE,
    cell GEOMETRY(POLYGON,4326) NOT NULL,
    centroid GEOGRAPHY(POINT,4326) NOT NULL,
    elevation_m INTEGER NOT NULL,
    slope_deg NUMERIC(5,2),
    aspect_deg SMALLINT CHECK (aspect_deg BETWEEN 0 AND 359),
    roughness NUMERIC(8,3),
    landcover_code TEXT,
    solar_gain_index NUMERIC(5,3),
    created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE geo.thermal_trigger_zones (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name TEXT,
    geometry GEOMETRY(POLYGON,4326) NOT NULL,
    source TEXT NOT NULL, -- terrain_model, pilot_observed, manual
    trigger_score NUMERIC(5,3) NOT NULL CHECK (trigger_score BETWEEN 0 AND 1),
    dominant_aspect_deg SMALLINT,
    min_sun_elevation_deg NUMERIC(5,2),
    notes TEXT,
    updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE TYPE geo.airspace_class AS ENUM
('A','B','C','D','E','F','G','CTR','TMA','RMZ','TMZ','RESTRICTED','PROHIBITED','DANGER','GLIDER','UNKNOWN');

CREATE TABLE geo.airspace_zones (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    source TEXT NOT NULL DEFAULT 'openaip',
    source_external_id TEXT,
    name TEXT NOT NULL,
    class geo.airspace_class NOT NULL,
    geometry GEOMETRY(MULTIPOLYGON,4326) NOT NULL,
    lower_limit TEXT NOT NULL, -- e.g. GND, 1500FT AMSL, FL95
    upper_limit TEXT NOT NULL,
    lower_altitude_m INTEGER,
    upper_altitude_m INTEGER,
    active_schedule JSONB DEFAULT '{}',
    rules TEXT,
    valid_from DATE,
    valid_to DATE,
    source_revision TEXT,
    updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX dem_tiles_bounds_gix ON geo.dem_tiles USING GIST(bounds);
CREATE INDEX terrain_cells_cell_gix ON geo.terrain_cells USING GIST(cell);
CREATE INDEX terrain_cells_centroid_gix ON geo.terrain_cells USING GIST(centroid);
CREATE INDEX terrain_cells_aspect_slope_idx ON geo.terrain_cells(aspect_deg, slope_deg);
CREATE INDEX thermal_trigger_geom_gix ON geo.thermal_trigger_zones USING GIST(geometry);
CREATE INDEX airspace_geom_gix ON geo.airspace_zones USING GIST(geometry);
CREATE INDEX airspace_class_idx ON geo.airspace_zones(class);
```

10. Weather Provider, Model Run and Forecast Schema

Weather data is split into provider metadata, model runs, grid forecasts, site-level forecasts and corrected forecasts. This prevents vendor lock-in and makes model comparison possible.

```
CREATE TABLE weather.providers (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    code TEXT UNIQUE NOT NULL, -- open_meteo, met_no, noaa_gfs, ecmwf
    name TEXT NOT NULL,
    license_name TEXT,
    attribution_text TEXT,
    base_url TEXT,
    requires_api_key BOOLEAN DEFAULT false,
```

```
    terms_url TEXT,
    created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE weather.weather_models (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    provider_id UUID NOT NULL REFERENCES weather.providers(id),
    code TEXT NOT NULL, -- gfs_0p25, ecmwf_ifs, met_no_nordic, icon_eu
    name TEXT NOT NULL,
    horizontal_resolution_km NUMERIC(6,2),
    vertical_levels JSONB DEFAULT '[]',
    forecast_horizon_hours INTEGER,
    update_frequency_minutes INTEGER,
    variables JSONB NOT NULL DEFAULT '[]',
    UNIQUE(provider_id, code)
);

CREATE TABLE weather.model_runs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    model_id UUID NOT NULL REFERENCES weather.weather_models(id),
    run_time TIMESTAMPTZ NOT NULL,
    horizon_start TIMESTAMPTZ NOT NULL,
    horizon_end TIMESTAMPTZ NOT NULL,
    status TEXT NOT NULL DEFAULT 'ingested', -- scheduled, downloading, ingested, normalized, failed
    raw_object_uri TEXT,
    checksum_sha256 TEXT,
    metadata JSONB DEFAULT '{}',
    ingested_at TIMESTAMPTZ DEFAULT now(),
    UNIQUE(model_id, run_time)
);

CREATE TABLE weather.grid_cells (
    id BIGSERIAL PRIMARY KEY,
    model_id UUID NOT NULL REFERENCES weather.weather_models(id),
    grid_hash TEXT NOT NULL,
    center GEOGRAPHY(POINT,4326) NOT NULL,
    cell GEOMETRY(POLYGON,4326),
    elevation_m INTEGER,
    UNIQUE(model_id, grid_hash)
);

CREATE TABLE weather.forecast_grid_hourly (
    time TIMESTAMPTZ NOT NULL,
    model_run_id UUID NOT NULL REFERENCES weather.model_runs(id),
    grid_cell_id BIGINT NOT NULL REFERENCES weather.grid_cells(id),
    pressure_level_hpa SMALLINT, -- NULL for surface/2m/10m values
    altitude_m INTEGER,          -- normalized altitude if known
    wind_u_ms NUMERIC(6,2),
    wind_v_ms NUMERIC(6,2),
    wind_speed_ms NUMERIC(6,2),
    wind_dir_deg SMALLINT,
    gust_ms NUMERIC(6,2),
    temp_c NUMERIC(5,2),
    dewpoint_c NUMERIC(5,2),
    relative_humidity_pct NUMERIC(5,2),
    pressure_hpa NUMERIC(7,2),
    cloud_cover_pct NUMERIC(5,2),
    precipitation_mm NUMERIC(7,3),
    cape_jkg NUMERIC(8,2),
    lifted_index NUMERIC(5,2),
    visibility_m INTEGER,
    raw JSONB DEFAULT '{}',
    PRIMARY KEY (time, model_run_id, grid_cell_id, COALESCE(pressure_level_hpa,0))
);

SELECT create_hypertable('weather.forecast_grid_hourly', 'time', if_not_exists => TRUE);
CREATE INDEX forecast_grid_run_time_idx ON weather.forecast_grid_hourly(model_run_id, time DESC);
CREATE INDEX forecast_grid_cell_time_idx ON weather.forecast_grid_hourly(grid_cell_id, time DESC);
CREATE INDEX forecast_grid_time_brin ON weather.forecast_grid_hourly USING BRIN(time);
```

Site-level forecast table used by the app API

```
CREATE TABLE weather.site_forecast_hourly (
    time TIMESTAMPTZ NOT NULL,
    launch_site_id UUID NOT NULL REFERENCES catalog.launch_sites(id) ON DELETE CASCADE,
    model_run_id UUID NOT NULL REFERENCES weather.model_runs(id),
    source_grid_cell_id BIGINT REFERENCES weather.grid_cells(id),
    interpolation_method TEXT NOT NULL DEFAULT 'nearest',
```

```
    wind_surface_ms NUMERIC(5,2),
    wind_surface_dir_deg SMALLINT,
    gust_surface_ms NUMERIC(5,2),
    wind_500m_ms NUMERIC(5,2),
    wind_500m_dir_deg SMALLINT,
    wind_1000m_ms NUMERIC(5,2),
    wind_1000m_dir_deg SMALLINT,
    wind_1500m_ms NUMERIC(5,2),
    wind_1500m_dir_deg SMALLINT,
    wind_2000m_ms NUMERIC(5,2),
    wind_2000m_dir_deg SMALLINT,
    cloudbase_msl_m INTEGER,
    cloudbase_agl_m INTEGER,
    thermal_strength_ms NUMERIC(4,2),
    thermal_top_msl_m INTEGER,
    precipitation_probability NUMERIC(4,3),
    precipitation_mm NUMERIC(7,3),
    thunderstorm_probability NUMERIC(4,3),
    visibility_m INTEGER,
    model_confidence NUMERIC(4,3),
    variables JSONB DEFAULT '{}',
    PRIMARY KEY (time, launch_site_id, model_run_id)
);

SELECT create_hypertable('weather.site_forecast_hourly', 'time', if_not_exists => TRUE);
CREATE INDEX site_forecast_site_time_idx ON weather.site_forecast_hourly(launch_site_id, time DESC);
CREATE INDEX site_forecast_run_idx ON weather.site_forecast_hourly(model_run_id);
CREATE INDEX site_forecast_cloudbase_idx ON weather.site_forecast_hourly(launch_site_id, cloudbase_agl_m);
```

Column	Implementation note
wind_500m/1000m/1500m/2000m	Normalized altitude bands required by paragliding UI altitude slider and XC planner.
cloudbase_agl_m	Calculated relative to launch altitude; important for launch/terrain safety.
thermal_strength_ms	May be model-provided in later versions or estimated by thermal engine.
model_confidence	Derived from model agreement, provider reliability and recency.
variables JSONB	Stores extended variables without schema churn, but core variables stay typed for indexes and fast queries.

11. Reality Layer: Station and Pilot Observation Schema

```
CREATE TABLE weather.weather_stations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    launch_site_id UUID REFERENCES catalog.launch_sites(id) ON DELETE SET NULL,
    provider TEXT NOT NULL,
    external_id TEXT,
    name TEXT NOT NULL,
    location GEOGRAPHY(POINT,4326) NOT NULL,
    altitude_m INTEGER,
    station_type TEXT, -- launch, landing, ridge, valley, airport, personal
    trust_score NUMERIC(4,3) DEFAULT 0.500,
    active BOOLEAN DEFAULT true,
    metadata JSONB DEFAULT '{}',
    UNIQUE(provider, external_id)
);

CREATE TABLE weather.station_observations (
    time TIMESTAMPTZ NOT NULL,
    station_id UUID NOT NULL REFERENCES weather.weather_stations(id) ON DELETE CASCADE,
    wind_ms NUMERIC(5,2),
    wind_dir_deg SMALLINT,
    gust_ms NUMERIC(5,2),
    temp_c NUMERIC(5,2),
    pressure_hpa NUMERIC(7,2),
    humidity_pct NUMERIC(5,2),
    precipitation_mm NUMERIC(7,3),
    raw JSONB DEFAULT '{}',
    PRIMARY KEY (time, station_id)
);
SELECT create_hypertable('weather.station_observations', 'time', if_not_exists => TRUE);
CREATE INDEX station_obs_station_time_idx ON weather.station_observations(station_id, time DESC);
CREATE INDEX station_obs_time_brin ON weather.station_observations USING BRIN(time);
```

```
CREATE TABLE weather.pilot_observations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES auth.users(id) ON DELETE SET NULL,
    launch_site_id UUID REFERENCES catalog.launch_sites(id) ON DELETE CASCADE,
    observed_at TIMESTAMPTZ NOT NULL,
    location GEOGRAPHY(POINT,4326),
    observation_type TEXT NOT NULL, -- launch_wind, cloudbase, thermal, turbulence, rain, hazard, landed
    wind_ms NUMERIC(5,2),
    wind_dir_deg SMALLINT,
    gust_ms NUMERIC(5,2),
    cloudbase_msl_m INTEGER,
    climb_ms NUMERIC(4,2),
    turbulence_level SMALLINT CHECK (turbulence_level BETWEEN 0 AND 5),
    confidence SMALLINT CHECK (confidence BETWEEN 1 AND 5),
    notes TEXT,
    reputation_weight NUMERIC(4,3) DEFAULT 0.500,
    media_uri TEXT,
    created_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX pilot_obs_site_time_idx ON weather.pilot_observations(launch_site_id, observed_at DESC);
CREATE INDEX pilot_obs_location_gix ON weather.pilot_observations USING GIST(location);
```

12. Forecast Correction and Digital Twin Schema

The digital twin layer is the competitive moat. It stores model errors, corrections and site-specific learning. Corrections are versioned and never overwrite raw forecasts.

```
CREATE TABLE weather.site_digital_twins (
    launch_site_id UUID PRIMARY KEY REFERENCES catalog.launch_sites(id) ON DELETE CASCADE,
    twin_version INTEGER NOT NULL DEFAULT 1,
    training_status TEXT NOT NULL DEFAULT 'cold_start', -- cold_start, training, active, degraded
    sample_count INTEGER NOT NULL DEFAULT 0,
    last_trained_at TIMESTAMPTZ,
    active_model_artifact_uri TEXT,
    active_model_hash TEXT,
    model_metrics JSONB DEFAULT '{}',
    notes TEXT,
    updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE weather.forecast_error_samples (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    launch_site_id UUID NOT NULL REFERENCES catalog.launch_sites(id) ON DELETE CASCADE,
    model_run_id UUID REFERENCES weather.model_runs(id),
    forecast_time TIMESTAMPTZ NOT NULL,
    observed_time TIMESTAMPTZ NOT NULL,
    variable TEXT NOT NULL, -- wind_ms, gust_ms, wind_dir_deg, cloudbase_msl_m, thermal_strength_ms
    forecast_value NUMERIC(10,3),
    observed_value NUMERIC(10,3),
    error_value NUMERIC(10,3),
    observation_source TEXT NOT NULL, -- station, pilot, track, manual_review
    quality_score NUMERIC(4,3) NOT NULL DEFAULT 0.500,
    feature_vector JSONB NOT NULL DEFAULT '{}',
    created_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX forecast_error_site_time_idx ON weather.forecast_error_samples(launch_site_id, forecast_time DESC);
CREATE INDEX forecast_error_var_idx ON weather.forecast_error_samples(variable, quality_score);
CREATE INDEX forecast_error_features_gin ON weather.forecast_error_samples USING GIN(feature_vector);

CREATE TABLE weather.corrected_site_forecast_hourly (
    time TIMESTAMPTZ NOT NULL,
    launch_site_id UUID NOT NULL REFERENCES catalog.launch_sites(id) ON DELETE CASCADE,
    base_model_run_id UUID NOT NULL REFERENCES weather.model_runs(id),
    correction_model_hash TEXT NOT NULL,
    wind_surface_ms NUMERIC(5,2),
    wind_surface_dir_deg SMALLINT,
    gust_surface_ms NUMERIC(5,2),
    cloudbase_msl_m INTEGER,
    cloudbase_agl_m INTEGER,
    thermal_strength_ms NUMERIC(4,2),
    uncertainty_low JSONB DEFAULT '{}',
    uncertainty_high JSONB DEFAULT '{}',
    correction_delta JSONB DEFAULT '{}',
    confidence NUMERIC(4,3),
```

```
    PRIMARY KEY (time, launch_site_id, base_model_run_id, correction_model_hash)
);
SELECT create_hypertable('weather.corrected_site_forecast_hourly', 'time', if_not_exists => TRUE);
CREATE INDEX corrected_site_forecast_site_time_idx ON weather.corrected_site_forecast_hourly(launch_site_id, time DESC);
```

13. Flyability, Risk and Decision Schema

Risk and recommendation tables must preserve not just the final score, but the reasons, hard blockers and input versions. This supports explainability and safety audits.

```
CREATE TYPE risk.decision_status AS ENUM ('GO','MAYBE','NO_GO','INSUFFICIENT_DATA');
CREATE TYPE risk.blocker_severity AS ENUM ('info','warning','hard_blocker');

CREATE TABLE risk.flyability_hourly (
    time TIMESTAMPTZ NOT NULL,
    launch_site_id UUID NOT NULL REFERENCES catalog.launch_sites(id) ON DELETE CASCADE,
    pilot_profile_hash TEXT NOT NULL DEFAULT 'generic',
    forecast_ref JSONB NOT NULL, -- model_run_id/correction_hash/source version references
    decision risk.decision_status NOT NULL,
    flyability_score SMALLINT NOT NULL CHECK (flyability_score BETWEEN 0 AND 100),
    safety_score SMALLINT NOT NULL CHECK (safety_score BETWEEN 0 AND 100),
    xc_score SMALLINT CHECK (xc_score BETWEEN 0 AND 100),
    confidence NUMERIC(4,3) NOT NULL,
    best_window_rank SMALLINT,
    summary TEXT,
    created_at TIMESTAMPTZ DEFAULT now(),
    PRIMARY KEY (time, launch_site_id, pilot_profile_hash)
);
SELECT create_hypertable('risk.flyability_hourly', 'time', if_not_exists => TRUE);
CREATE INDEX flyability_site_time_idx ON risk.flyability_hourly(launch_site_id, time DESC);
CREATE INDEX flyability_decision_idx ON risk.flyability_hourly(decision, flyability_score DESC);

CREATE TABLE risk.risk_factor_values (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    time TIMESTAMPTZ NOT NULL,
    launch_site_id UUID NOT NULL REFERENCES catalog.launch_sites(id) ON DELETE CASCADE,
    pilot_profile_hash TEXT NOT NULL DEFAULT 'generic',
    factor_code TEXT NOT NULL, -- wind_speed, gust_spread, wind_direction, cloudbase, rotor, foehn, thunderstorm
    raw_value NUMERIC(10,3),
    normalized_score SMALLINT CHECK (normalized_score BETWEEN 0 AND 100),
    weight NUMERIC(5,4) NOT NULL,
    severity risk.blocker_severity NOT NULL DEFAULT 'info',
    reason TEXT NOT NULL,
    evidence JSONB DEFAULT '{}'
);
CREATE INDEX risk_factor_lookup_idx ON risk.risk_factor_values(launch_site_id, time, pilot_profile_hash);
CREATE INDEX risk_factor_code_idx ON risk.risk_factor_values(factor_code, severity);

CREATE TABLE risk.hard_blockers (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    code TEXT UNIQUE NOT NULL,
    description TEXT NOT NULL,
    default_threshold JSONB NOT NULL,
    enabled BOOLEAN DEFAULT true
);

CREATE TABLE risk.decision_snapshots (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    launch_site_id UUID NOT NULL REFERENCES catalog.launch_sites(id),
    user_id UUID REFERENCES auth.users(id),
    requested_for TIMESTAMPTZ NOT NULL,
    decision risk.decision_status NOT NULL,
    input_context JSONB NOT NULL,
    score_breakdown JSONB NOT NULL,
    model_versions JSONB NOT NULL,
    created_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX decision_snapshots_site_time_idx ON risk.decision_snapshots(launch_site_id, requested_for DESC);
CREATE INDEX decision_snapshots_context_gin ON risk.decision_snapshots USING GIN(input_context);
```

Hard blocker	Example DB condition
Thunderstorm risk	thunderstorm_probability >= 0.25 OR lightning within configured radius.
Excessive gust spread	gust_surface_ms - wind_surface_ms above launch orientation threshold.

Wrong launch direction	wind direction outside any active launch orientation sector.
Low cloudbase	cloudbase_agl_m lower than site min or local terrain clearance rule.
Strong rotor/föhn/wave risk	risk factor severity = hard_blocker from terrain/weather model.
Insufficient data	no fresh model run, missing critical variable, or data quality check failed.

14. XC Planner and Route Data Schema

```
CREATE TABLE flight.landing_fields (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name TEXT,
    location GEOGRAPHY(POINT,4326) NOT NULL,
    area GEOMETRY(POLYGON,4326),
    altitude_m INTEGER,
    suitability_score NUMERIC(4,3) DEFAULT 0.500,
    hazard_notes TEXT,
    source TEXT NOT NULL DEFAULT 'community',
    updated_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX landing_fields_location_gix ON flight.landing_fields USING GIST(location);
CREATE INDEX landing_fields_area_gix ON flight.landing_fields USING GIST(area);

CREATE TABLE flight.xc_route_plans (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES auth.users(id),
    launch_site_id UUID NOT NULL REFERENCES catalog.launch_sites(id),
    requested_for TIMESTAMPTZ NOT NULL,
    target_distance_km NUMERIC(7,2),
    route_geometry GEOMETRY(LINESTRING,4326),
    expected_distance_km NUMERIC(7,2),
    expected_duration_minutes INTEGER,
    expected_max_altitude_m INTEGER,
    expected_min_cloudbase_m INTEGER,
    risk_summary JSONB DEFAULT '{}',
    plan_status TEXT DEFAULT 'generated',
    created_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX xc_route_plans_launch_time_idx ON flight.xc_route_plans(launch_site_id, requested_for DESC);
CREATE INDEX xc_route_geom_gix ON flight.xc_route_plans USING GIST(route_geometry);

CREATE TABLE flight.xc_route_segments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    route_plan_id UUID NOT NULL REFERENCES flight.xc_route_plans(id) ON DELETE CASCADE,
    segment_order INTEGER NOT NULL,
    segment_geometry GEOMETRY(LINESTRING,4326) NOT NULL,
    segment_type TEXT NOT NULL, -- climb, glide, ridge, valley_crossing, final_glide
    expected_climb_ms NUMERIC(4,2),
    expected_tailwind_ms NUMERIC(4,2),
    min_arrival_altitude_m INTEGER,
    notes TEXT,
    UNIQUE(route_plan_id, segment_order)
);
CREATE INDEX xc_route_segments_geom_gix ON flight.xc_route_segments USING GIST(segment_geometry);
```

15. Flight Tracking and IGC Schema

```
CREATE TABLE flight.tracks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES auth.users(id) ON DELETE SET NULL,
    launch_site_id UUID REFERENCES catalog.launch_sites(id) ON DELETE SET NULL,
    landing_field_id UUID REFERENCES flight.landing_fields(id) ON DELETE SET NULL,
    started_at TIMESTAMPTZ NOT NULL,
    ended_at TIMESTAMPTZ,
    duration_seconds INTEGER,
    distance_km NUMERIC(8,2),
    max_altitude_m INTEGER,
    max_climb_ms NUMERIC(4,2),
    source TEXT NOT NULL DEFAULT 'mobile', -- mobile, igc_upload, device_import
    igc_object_uri TEXT,
    privacy_level TEXT DEFAULT 'private',
    created_at TIMESTAMPTZ DEFAULT now()
```

```
);
CREATE INDEX tracks_user_time_idx ON flight.tracks(user_id, started_at DESC);
CREATE INDEX tracks_site_time_idx ON flight.tracks(launch_site_id, started_at DESC);

CREATE TABLE flight.track_points (
    time TIMESTAMPTZ NOT NULL,
    track_id UUID NOT NULL REFERENCES flight.tracks(id) ON DELETE CASCADE,
    point GEOGRAPHY(POINT,4326) NOT NULL,
    altitude_gps_m INTEGER,
    altitude_pressure_m INTEGER,
    ground_speed_ms NUMERIC(5,2),
    climb_ms NUMERIC(4,2),
    heading_deg SMALLINT,
    accuracy_m NUMERIC(6,2),
    battery_pct SMALLINT,
    raw JSONB DEFAULT '{}',
    PRIMARY KEY (time, track_id)
);
SELECT create_hypertable('flight.track_points', 'time', if_not_exists => TRUE);
CREATE INDEX track_points_track_time_idx ON flight.track_points(track_id, time DESC);
CREATE INDEX track_points_point_gix ON flight.track_points USING GIST(point);
CREATE INDEX track_points_time_brin ON flight.track_points USING BRIN(time);

CREATE TABLE flight.live_sessions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
    track_id UUID REFERENCES flight.tracks(id) ON DELETE SET NULL,
    status TEXT NOT NULL DEFAULT 'active', -- active, landed, stale, emergency
    last_point GEOGRAPHY(POINT,4326),
    last_altitude_m INTEGER,
    last_seen_at TIMESTAMPTZ,
    emergency_enabled BOOLEAN DEFAULT false,
    share_token_hash TEXT,
    created_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX live_sessions_status_idx ON flight.live_sessions(status, last_seen_at DESC);
CREATE INDEX live_sessions_last_point_gix ON flight.live_sessions USING GIST(last_point);
```

16. AI Briefing and Grounding Schema

AI answers must be grounded in stored weather and risk context. The UI can display natural language, but the DB stores structured inputs, model versions and safety disclaimers.

```
CREATE TABLE ai.briefing_requests (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES auth.users(id) ON DELETE SET NULL,
    launch_site_id UUID REFERENCES catalog.launch_sites(id),
    request_text TEXT NOT NULL,
    requested_for_start TIMESTAMPTZ,
    requested_for_end TIMESTAMPTZ,
    request_type TEXT NOT NULL, -- site_briefing, where_to_fly, xc_plan, debrief
    language TEXT DEFAULT 'en',
    created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE ai.briefing_context_snapshots (
    request_id UUID PRIMARY KEY REFERENCES ai.briefing_requests(id) ON DELETE CASCADE,
    weather_context JSONB NOT NULL,
    risk_context JSONB NOT NULL,
    site_context JSONB NOT NULL,
    pilot_context JSONB NOT NULL,
    entitlement_context JSONB NOT NULL,
    source_refs JSONB NOT NULL,
    context_hash TEXT NOT NULL,
    created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE ai.briefing_outputs (
    request_id UUID PRIMARY KEY REFERENCES ai.briefing_requests(id) ON DELETE CASCADE,
    llm_provider TEXT NOT NULL,
    llm_model TEXT NOT NULL,
    prompt_template_version TEXT NOT NULL,
    answer_text TEXT NOT NULL,
    structured_answer JSONB NOT NULL,
```

```
    safety_disclaimer_included BOOLEAN NOT NULL DEFAULT true,
    hallucination_guard_passed BOOLEAN NOT NULL DEFAULT false,
    output_hash TEXT NOT NULL,
    token_usage JSONB DEFAULT '{}',
    created_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX briefing_requests_user_time_idx ON ai.briefing_requests(user_id, created_at DESC);
CREATE INDEX briefing_outputs_structured_gin ON ai.briefing_outputs USING GIN(structured_answer);
-- AI structured answer example stored in ai.briefing_outputs.structured_answer
{
  "status": "MAYBE",
  "best_window": {"start": "2026-06-05T11:00:00Z", "end": "2026-06-05T14:30:00Z"},
  "main_risks": ["gust spread rising after 15:00", "cloudbase marginal for XC"],
  "hard_blockers": [],
  "confidence": 0.74,
  "source_forecast_ids": ["site_forecast_hourly:..."],
  "do_not_invent_weather": true
}
```

17. Alert Rules, Notification and Outbox Schema

```
CREATE TABLE alerts.alert_rules (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
    launch_site_id UUID REFERENCES catalog.launch_sites(id) ON DELETE CASCADE,
    rule_name TEXT NOT NULL,
    enabled BOOLEAN NOT NULL DEFAULT true,
    rule_type TEXT NOT NULL, -- flyable_window, danger_change, cloudbase, gust, thunderstorm, airspace, custom
    conditions JSONB NOT NULL,
    channels JSONB NOT NULL DEFAULT '["push"]',
    quiet_hours JSONB DEFAULT '{}',
    last_triggered_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX alert_rules_user_idx ON alerts.alert_rules(user_id, enabled);
CREATE INDEX alert_rules_site_idx ON alerts.alert_rules(launch_site_id, enabled);
CREATE INDEX alert_rules_conditions_gin ON alerts.alert_rules USING GIN(conditions);

CREATE TABLE alerts.alert_events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    rule_id UUID REFERENCES alerts.alert_rules(id) ON DELETE SET NULL,
    user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
    launch_site_id UUID REFERENCES catalog.launch_sites(id),
    event_type TEXT NOT NULL,
    severity TEXT NOT NULL DEFAULT 'info',
    payload JSONB NOT NULL,
    dedupe_key TEXT,
    status TEXT NOT NULL DEFAULT 'pending', -- pending, sent, suppressed, failed
    created_at TIMESTAMPTZ DEFAULT now()
);
CREATE UNIQUE INDEX alert_events_dedupe_idx ON alerts.alert_events(dedupe_key) WHERE dedupe_key IS NOT NULL;

CREATE TABLE alerts.notification_attempts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    alert_event_id UUID NOT NULL REFERENCES alerts.alert_events(id) ON DELETE CASCADE,
    channel TEXT NOT NULL, -- push, email, sms, webhook
    provider TEXT,
    status TEXT NOT NULL,
    provider_message_id TEXT,
    error_message TEXT,
    attempted_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE ops.outbox_events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    aggregate_type TEXT NOT NULL,
    aggregate_id UUID NOT NULL,
    event_type TEXT NOT NULL,
    payload JSONB NOT NULL,
    status TEXT NOT NULL DEFAULT 'pending',
    created_at TIMESTAMPTZ DEFAULT now(),
    published_at TIMESTAMPTZ
);
CREATE INDEX outbox_pending_idx ON ops.outbox_events(status, created_at) WHERE status = 'pending';
```

18. Paid Tier, Entitlement and Usage Schema

```
CREATE TABLE billing.plans (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    code TEXT UNIQUE NOT NULL, -- FREE, PRO, PRO_PLUS, CLUB, SCHOOL, EVENT, API
    name TEXT NOT NULL,
    billing_period TEXT NOT NULL, -- monthly, yearly, event, enterprise
    price_cents INTEGER NOT NULL DEFAULT 0,
    currency CHAR(3) NOT NULL DEFAULT 'EUR',
    active BOOLEAN DEFAULT true,
    public_features JSONB NOT NULL DEFAULT '[]'
);

CREATE TABLE billing.entitlements (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    plan_id UUID NOT NULL REFERENCES billing.plans(id) ON DELETE CASCADE,
    key TEXT NOT NULL, -- forecast_days, ai_briefings_monthly, advanced_layers, model_comparison, api_calls_monthly
    value JSONB NOT NULL,
    UNIQUE(plan_id, key)
);

CREATE TABLE billing.subscriptions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
    organization_id UUID,
    plan_id UUID NOT NULL REFERENCES billing.plans(id),
    status TEXT NOT NULL, -- trialing, active, past_due, canceled, comped
    provider TEXT NOT NULL DEFAULT 'stripe',
    provider_subscription_id TEXT,
    current_period_start TIMESTAMPTZ,
    current_period_end TIMESTAMPTZ,
    created_at TIMESTAMPTZ DEFAULT now(),
    canceled_at TIMESTAMPTZ
);
CREATE INDEX subscriptions_user_status_idx ON billing.subscriptions(user_id, status);

CREATE TABLE billing.usage_meter (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    subject_type TEXT NOT NULL, -- user, org, api_key
    subject_id UUID NOT NULL,
    usage_key TEXT NOT NULL, -- ai_briefing, api_call, map_tile, export, route_plan
    period_start TIMESTAMPTZ NOT NULL,
    period_end TIMESTAMPTZ NOT NULL,
    quantity BIGINT NOT NULL DEFAULT 0,
    metadata JSONB DEFAULT '{}',
    UNIQUE(subject_type, subject_id, usage_key, period_start)
);
CREATE INDEX usage_meter_subject_idx ON billing.usage_meter(subject_type, subject_id, usage_key, period_start DESC);
```

Entitlement key	Example value	Feature gate
forecast_days	{"days": 3} / {"days": 10}	Limits forecast horizon visible to user.
advanced_layers	{"enabled": true}	Thermals, model comparison, wave, rotor, valley wind.
ai_briefings_monthly	{"count": 50}	AI briefing quota.
favorite_sites	{"count": 5} / {"unlimited": true}	Favorites limit.
api_calls_monthly	{"count": 100000}	API subscription tier.
org_members	{"count": 50}	Club/school member count.

19. Club, School and Event Schema

```
CREATE TYPE org.organization_type AS ENUM ('club','school','tandem_operator','rescue','event_organizer','enterprise');

CREATE TABLE org.organizations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name TEXT NOT NULL,
    type org.organization_type NOT NULL,
    country_code CHAR(2),
    home_location GEOGRAPHY(POINT,4326),
    billing_owner_user_id UUID REFERENCES auth.users(id),
    settings JSONB DEFAULT '{}',
    created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE org.memberships (
```

```
    organization_id UUID NOT NULL REFERENCES org.organizations(id) ON DELETE CASCADE,
    user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
    role TEXT NOT NULL, -- owner, admin, instructor, pilot, student, safety_officer
    status TEXT NOT NULL DEFAULT 'active',
    joined_at TIMESTAMPTZ DEFAULT now(),
    PRIMARY KEY (organization_id, user_id)
);

CREATE TABLE org.school_students (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID NOT NULL REFERENCES org.organizations(id) ON DELETE CASCADE,
    user_id UUID REFERENCES auth.users(id) ON DELETE SET NULL,
    instructor_user_id UUID REFERENCES auth.users(id) ON DELETE SET NULL,
    training_level TEXT NOT NULL,
    restrictions JSONB DEFAULT '{}',
    progress JSONB DEFAULT '{}',
    active BOOLEAN DEFAULT true,
    created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE org.events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id UUID REFERENCES org.organizations(id) ON DELETE SET NULL,
    name TEXT NOT NULL,
    event_type TEXT NOT NULL, -- competition, festival, safety_day
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    primary_area GEOMETRY(POLYGON,4326),
    rules JSONB DEFAULT '{}',
    created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE org.event_tasks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    event_id UUID NOT NULL REFERENCES org.events(id) ON DELETE CASCADE,
    name TEXT NOT NULL,
    task_date DATE NOT NULL,
    route_geometry GEOMETRY(LINESTRING,4326),
    turnpoints JSONB NOT NULL DEFAULT '[]',
    weather_briefing_snapshot JSONB,
    status TEXT DEFAULT 'draft',
    created_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX organizations_home_gix ON org.organizations USING GIST(home_location);
CREATE INDEX events_area_gix ON org.events USING GIST(primary_area);
CREATE INDEX event_tasks_route_gix ON org.event_tasks USING GIST(route_geometry);
```

20. Operations, Data Quality and Audit Schema

```
CREATE TABLE ops.ingestion_jobs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    provider_id UUID REFERENCES weather.providers(id),
    model_id UUID REFERENCES weather.weather_models(id),
    model_run_id UUID REFERENCES weather.model_runs(id),
    job_type TEXT NOT NULL, -- download, parse, normalize, interpolate, score, correct
    status TEXT NOT NULL DEFAULT 'queued',
    started_at TIMESTAMPTZ,
    finished_at TIMESTAMPTZ,
    attempt INTEGER NOT NULL DEFAULT 1,
    records_in BIGINT DEFAULT 0,
    records_out BIGINT DEFAULT 0,
    error_message TEXT,
    metadata JSONB DEFAULT '{}',
    created_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX ingestion_jobs_status_idx ON ops.ingestion_jobs(status, created_at DESC);
CREATE INDEX ingestion_jobs_model_run_idx ON ops.ingestion_jobs(model_run_id, job_type);

CREATE TABLE ops.data_quality_checks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    check_name TEXT NOT NULL,
    scope TEXT NOT NULL, -- provider, model_run, site, table, service
    scope_id TEXT,
    severity TEXT NOT NULL, -- info, warning, critical
    status TEXT NOT NULL, -- pass, fail, skipped
```

```
    measured_value JSONB DEFAULT '{}',
    threshold JSONB DEFAULT '{}',
    created_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX dq_checks_scope_idx ON ops.data_quality_checks(scope, scope_id, created_at DESC);
CREATE INDEX dq_checks_fail_idx ON ops.data_quality_checks(status, severity, created_at DESC) WHERE status = 'fail';

CREATE TABLE audit.audit_log (
    id BIGSERIAL PRIMARY KEY,
    actor_user_id UUID,
    actor_service TEXT,
    action TEXT NOT NULL,
    entity_type TEXT NOT NULL,
    entity_id TEXT,
    ip_address INET,
    user_agent TEXT,
    before_state JSONB,
    after_state JSONB,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX audit_log_actor_time_idx ON audit.audit_log(actor_user_id, created_at DESC);
CREATE INDEX audit_log_entity_idx ON audit.audit_log(entity_type, entity_id);
```

21. Retention, Archiving and Storage Lifecycle

Dataset	Hot retention	Cold retention	Reason
weather.forecast_grid_hourly	90-180 days	Object storage raw GRIB/NetCDF for 1-5 years depending tier	Grid data is heavy; site forecasts and error samples preserve high-value derived data.
weather.site_forecast_hourly	2 years	Aggregated summaries indefinitely	Needed for forecast verification and user history.
weather.station_observations	5 years	Compressed historical hypertables	Important for correction model training.
weather.pilot_observations	5 years	Anonymized long-term aggregate	High-value reality signal.
flight.track_points	User-controlled; default 3 years private	Anonymized aggregate if consented	Tracks are sensitive location data.
risk.flyability_hourly	5 years	Aggregated climatology indefinitely	Needed for product metrics and model validation.
ai.briefing_context_snapshots	12 months default	Hash + structured safety metrics retained	Avoid storing too much AI conversation data.
billing/audit	Legal/accounting period	Archive per jurisdiction	Compliance and fraud investigation.

```
-- Example retention policies. Adjust per tier and compliance constraints.
SELECT add_retention_policy('weather.forecast_grid_hourly', INTERVAL '180 days', if_not_exists => TRUE);
SELECT add_retention_policy('weather.site_forecast_hourly', INTERVAL '730 days', if_not_exists => TRUE);
SELECT add_retention_policy('flight.track_points', INTERVAL '3 years', if_not_exists => TRUE);

-- Optional compression for older Timescale chunks.
ALTER TABLE weather.station_observations SET (timescaledb.compress, timescaledb.compress_segmentby = 'station_id');
SELECT add_compression_policy('weather.station_observations', INTERVAL '30 days', if_not_exists => TRUE);
```

22. Indexing and Query Performance Strategy

Query pattern	Index strategy
Nearest launches from user	GiST on catalog.launch_sites.launch_point geography.
Weather timeline for one launch	Composite B-tree launch_site_id + time DESC on site forecast and flyability hypertables.
Live airspace warning	GiST on geo.airspace_zones.geometry and track_points.point. Use bounding-box prefilter before altitude/schedule checks.
Large time range analytics	BRIN on time columns for hypertables.
Search site by name	GIN trigram index on launch site name.
AI/risk JSON context search	GIN on JSONB context/snapshot columns only where needed.
Billing usage quota check	Unique period index on subject, usage key and period start.
Outbox processing	Partial index on pending events.

```
-- Materialized view for fast home screen: best windows by site for the next 72 hours.
CREATE MATERIALIZED VIEW risk.site_best_windows_72h AS
SELECT launch_site_id,
```

```
        min(time) FILTER (WHERE decision = 'GO') AS first_go_time,
        max(flyability_score) AS best_score,
        avg(confidence) AS avg_confidence,
        jsonb_agg(jsonb_build_object('time', time, 'decision', decision, 'score', flyability_score)
                  ORDER BY time) AS timeline
FROM risk.flyability_hourly
WHERE time >= now() AND time < now() + interval '72 hours'
GROUP BY launch_site_id;

CREATE UNIQUE INDEX site_best_windows_72h_pk ON risk.site_best_windows_72h(launch_site_id);
```

23. API-to-Database Query Mapping

API endpoint	Primary DB objects	Notes
GET /v1/launches/nearby	catalog.launch_sites, risk.site_best_windows_72h	Uses PostGIS DWithin and optional flyability filter.
GET /v1/launches/{id}/forecast	weather.site_forecast_hourly, weather.corrected_site_forecast_hourly	Entitlement decides raw, corrected, model comparison and horizon.
GET /v1/launches/{id}/flyability	risk.flyability_hourly, risk.risk_factor_values	Returns decision, score and factor explanations.
POST /v1/briefings	ai.briefing_requests, ai.briefing_context_snapshots, ai.briefing_outputs	Creates auditable grounded AI response.
POST /v1/tracks/points	flight.live_sessions, flight.track_points	High write-rate endpoint; batch inserts preferred.
GET /v1/airspace/warnings	geo.airspace_zones, flight.live_sessions	Spatial intersection + altitude + active schedule.
POST /v1/alerts/rules	alerts.alert_rules	JSONB conditions validated by service schema.
GET /v1/billing/entitlements	billing.subscriptions, billing.entitlements	Backend gate; client flags are not trusted.

24. Migration, Seeding and Environment Strategy

- Use Alembic for schema migrations. Every migration must include forward migration, rollback and data backfill plan when applicable.
- Seed reference data for billing plans, default hard blockers, weather providers, core model definitions and public airspace classes.
- Use separate databases for local, staging, production and analytics replicas. Never run exploratory GIS or ML training queries directly on production primaries.
- Use read replicas for map browsing, public API and dashboards; keep write-heavy ingestion on primary plus partition-aware batch loaders.
- Schema migrations affecting hypertables must be load-tested against realistic row counts before production deployment.
- For high-volume insert workloads, use COPY or batch inserts, not one-row-per-request database writes.

```
-- Example seed: hard blockers
INSERT INTO risk.hard_blockers(code, description, default_threshold) VALUES
('THUNDERSTORM_PROBABILITY', 'Thunderstorm probability exceeds safe threshold', '{"max_probability": 0.25}'),
('GUST_SPREAD', 'Gust spread exceeds pilot/site safety threshold', '{"max_spread_ms": 4.0}'),
('WRONG_WIND_DIRECTION', 'Wind direction outside supported launch sector', '{"allowed_sector_required": true}'),
('LOW_CLOUDBASE', 'Cloudbase too low for terrain clearance', '{"min_cloudbase_agl_m": 300}'),
('STALE_FORECAST', 'No fresh forecast available', '{"max_model_age_hours": 8}')
ON CONFLICT (code) DO NOTHING;
```

25. Security, Privacy and Access Control Design

Data class	Tables	Controls
Public/low sensitivity	catalog.launch_sites, landing_zones, public site hazards	Public read APIs with moderation controls.
Operational weather	weather forecasts, risk scores	Read gated by tier; write only from ingestion/scoring services.
Sensitive location	flight.track_points, live_sessions, emergency state	Private by default, consent-based sharing, retention controls, audit access.
PII	auth.users, emergency contacts, billing subscriptions	Encrypted backups, RBAC, audit logging, deletion/anonymization workflow.
AI context	ai.briefing_context_snapshots and outputs	Avoid unnecessary PII; store source refs and

		context hash; limited retention.
Financial	billing subscriptions, usage meter	Provider IDs stored; card data must stay with payment provider.

```
-- Example row-level security direction for private tracks.
ALTER TABLE flight.tracks ENABLE ROW LEVEL SECURITY;
-- Application should set app.current_user_id at session level.
CREATE POLICY track_owner_read ON flight.tracks
FOR SELECT USING (user_id::text = current_setting('app.current_user_id', true));
```

26. Data Quality Rules and Validation Checks

Check	Failure action
Model run completeness below threshold	Mark model_run.status = failed or degraded; do not generate premium recommendation from incomplete run.
Wind speed impossible/outlier	Reject record or flag with data quality issue.
Forecast older than allowed model age	Risk engine returns INSUFFICIENT_DATA or degraded confidence.
Station observation stale	Exclude from correction sample generation.
Pilot observation low reputation or conflicting	Downweight; do not use as single hard truth.
Airspace source revision changed	Rebuild affected tile/cache and re-run safety tests.
DEM tile checksum mismatch	Do not process terrain cells; quarantine object.

27. Critical Data Workflows

27.1 Forecast ingestion to flyability

1. weather-ingestion-service creates a weather.model_runs record with status = downloading.
2. Raw GRIB/API payload is stored in object storage and checksum is written to weather.model_runs.
3. Parser normalizes grid cells and writes weather.forecast_grid_hourly with batch inserts.
4. Site interpolation job writes weather.site_forecast_hourly for active launch sites.
5. Digital twin job writes weather.corrected_site_forecast_hourly where trained corrections exist.
6. Risk engine reads site forecast + launch rules + pilot profile thresholds and writes risk.flyability_hourly and risk.risk_factor_values.
7. Alert engine compares new scores to user alert rules and writes alerts.alert_events.
8. Outbox publishes cache invalidation and notification messages.

27.2 Live flight to airspace warning

9. Mobile app batches GPS points to tracking-service.
10. tracking-service writes flight.track_points and updates flight.live_sessions.last_point.
11. airspace-service queries geo.airspace_zones using spatial intersection and altitude limits.
12. If predicted breach is detected, alert-service creates immediate push/voice warning event.
13. Audit record is written for emergency-level alerts.

28. Analytics and ML Feature Mart Design

Production OLTP tables should not be abused as the ML training warehouse. Create controlled feature marts from validated snapshots.

```
CREATE SCHEMA IF NOT EXISTS ml;

CREATE TABLE ml.site_hourly_feature_mart (
    feature_time TIMESTAMPTZ NOT NULL,
    launch_site_id UUID NOT NULL,
    model_run_id UUID,
    forecast_age_hours NUMERIC(5,2),
    wind_surface_ms NUMERIC(5,2),
    gust_surface_ms NUMERIC(5,2),
    wind_dir_deg SMALLINT,
    cloudbase_agl_m INTEGER,
    thermal_strength_ms NUMERIC(4,2),
```

```
    cape_jkg NUMERIC(8,2),
    terrain_aspect_deg SMALLINT,
    slope_deg NUMERIC(5,2),
    station_error_wind_ms NUMERIC(6,2),
    pilot_report_count_3h INTEGER,
    target_observed_wind_ms NUMERIC(5,2),
    target_observed_gust_ms NUMERIC(5,2),
    target_flyable BOOLEAN,
    data_version TEXT NOT NULL,
    created_at TIMESTAMPTZ DEFAULT now(),
    PRIMARY KEY (feature_time, launch_site_id, data_version)
);
SELECT create_hypertable('ml.site_hourly_feature_mart', 'feature_time', if_not_exists => TRUE);
```

29. MVP Database Slice

MVP feature	Minimum tables required
Launch database	auth.users, catalog.launch_sites, catalog.launch_orientations, catalog.landing_zones, catalog.site_hazards
Basic forecast	weather.providers, weather.weather_models, weather.model_runs, weather.site_forecast_hourly
Flyability score	risk.flyability_hourly, risk.risk_factor_values, risk.hard_blockers
Favorites/alerts	alerts.alert_rules, alerts.alert_events, alerts.notification_attempts
Paid Free/Pro gating	billing.plans, billing.entitlements, billing.subscriptions, billing.usage_meter
AI briefing MVP	ai.briefing_requests, ai.briefing_context_snapshots, ai.briefing_outputs
Basic tracking	flight.tracks, flight.track_points, flight.live_sessions
Operations	ops.ingestion_jobs, ops.data_quality_checks, audit.audit_log

30. Database Engineering Backlog

Priority	Task	Acceptance criteria
P0	Create schemas/extensions and MVP tables	All migrations run on clean DB and staging DB; pg_dump/restore verified.
P0	Implement launch site seed importer	Can import 1000+ launch sites with orientations and hazards.
P0	Implement site forecast hypertable	Can write/read 72-hour hourly forecast for 5000 sites under target latency.
P0	Implement flyability score tables	Risk engine stores reproducible scores and factor explanations.
P1	Add terrain/airspace schema	Can import OpenAIP zones and DEM-derived terrain features for target regions.
P1	Add correction/digital twin tables	Can store forecast error samples and corrected forecast outputs.
P1	Add tracking tables and live sessions	Can ingest batched GPS points and query active sessions.
P2	Add ML feature mart	Training data is point-in-time safe and versioned.
P2	Add analytics materialized views	Product dashboards run without impacting ingestion performance.

31. Technical References

- [1] PostGIS Manual - https://postgis.net/docs/
- [2] PostGIS geometry/geography FAQ - https://postgis.net/documentation/faq/geometry-or-geography/
- [3] PostgreSQL CREATE INDEX documentation - https://www.postgresql.org/docs/current/sql-createindex.html
- [4] PostgreSQL index types - https://www.postgresql.org/docs/current/indexes-types.html
- [5] Timescale/TigerData retention policy documentation - https://www.tigerdata.com/docs/build/data-management/data-retention/create-a-retention-policy
- [6] Open-Meteo Forecast API documentation - https://open-meteo.com/en/docs
- [7] MET Norway Locationforecast API documentation - https://api.met.no/weatherapi/locationforecast/2.0/documentation
- [8] NOAA NOMADS GFS data catalogue - https://nomads.ncep.noaa.gov/
- [9] OpenAIP API documentation - https://docs.openaip.net/

- [10] Copernicus DEM collection / API access - https://dataspace.copernicus.eu/explore-data/data-collections/copernicus-contributing-missions/collections-description/COP-DEM